E-Commerce Data Integration & Analytics Pipeline

1. Project Proposal:

Overview of the project

E-commerce companies often collect product, customer, order, and transaction data from multiple sources, such as REST APIs and CSV/JSON files. These datasets may contain duplicate records, missing values, inconsistent formats, and updated information, making it difficult to generate accurate and reliable sales reports.

This project aims to develop an automated end-to-end data engineering pipeline that integrates data from multiple sources, validates and transforms the data, stores it in a relational database, and organizes it into a star schema for analytical reporting.

Objectives

1. **Collect Data:** Read product data from an API, and customer/order data from CSV and JSON files.
2. **Clean Data:** Remove duplicate rows, fix missing numbers, and make all dates look the same.
3. **Store Data:** Put the clean data into a Star Schema database with 1 Fact table (`Fact_Sales`) and 3 Dimension tables (`Dim_Customer`, `Dim_Product`, `Dim_Date`).
4. **Automate:** Use Apache Airflow to run the whole process automatically on a schedule.

Key Features / Scope

- Relational database design and implementation.
- Star-schema development.
- Append-based or incremental loading.
- SQL views and indexes.
- Airflow orchestration, logging, retries, and error handling.
- Automated testing and data-quality checks.
- GitHub repository, technical documentation, architecture diagram, and execution screenshots.

Project Plan:

Week	Step Name	What We Will Do	Main Deliverable
Week 1	**M1:Design & Plan**	Choose the tools, draw the database tables, write project documentation.	Project Plan & Table Designs
Week 2	**M2: ETL Pipeline Core**	Write Python scripts to read API/CSV files and clean data.	Python Cleaning Scripts
Week 3	**M3: Data Warehouse**	Create PostgreSQL tables, load cleaned data, write SQL report queries.	Working Database & Views
Week 4	**M4: Automated Pipeline**	Set up Apache Airflow to run all Python scripts automatically in order.	Working Airflow Pipeline (DAG)
Week 5	**M5: Testing & Final Submission**	Test everything, put code on GitHub, make presentation slides and video demo.	Final GitHub Code & Slides

Deliverables

D1: Technical Documentation (This Document)

- Project proposal, schedule, requirements, risk assessment, and task roles.

D2: Database Setup Scripts (`.sql`) & ERD Diagram

- `schema.sql`: Code to create the tables (`Fact_Sales`, `Dim_Customer`, `Dim_Product`, `Dim_Date`).
- `views.sql`: Code for common analytical queries (e.g., daily sales summary).
- `ERD_Diagram.png`: Visual picture showing how the database tables connect.

D3: Python Data Processing Modules (`.py`)

- `ingest.py`: Script to download data from the REST API and read CSV/JSON files.
- `transform.py`: Script to clean missing values, drop duplicate rows, and fix dates using Pandas.

D4: Apache Airflow Workflow Script (`.py`)

- `ecommerce_etl_dag.py`: The automated Airflow schedule file with logging and retry settings.

D5: Data Quality Test Script (`.py`)

- `data_checks.py`: Script that verifies no blank primary keys or duplicate rows enter the database.

D6: Project GitHub Repository & Final README

- A shared GitHub folder containing all the code, a simple `README.md` guide explaining how to run the project, and screenshots of successful Airflow runs.

Task Assignment & Roles

Data Engineer 1 (Pipeline Lead):

- **Task:** Sets up Apache Airflow and creates the automated workflow (DAG) that runs all scripts in order.

Data Engineer 2 (Data Ingestion Specialist):

- **Task:** Writes simple Python code to fetch product data from the web API and load raw CSV/JSON files.

Data Engineer 3 (Data Transformation Lead):

- **Task:** Writes Python code (using Pandas) to delete duplicate rows, fix blank fields, and match customer/order records.

Data Engineer 4 (Database & SQL Specialist):

- **Task:** Writes SQL scripts to create the 4 database tables (`Fact_Sales`, `Dim_Customer`, `Dim_Product`, `Dim_Date`) and SQL views.

Data Engineer 5 (Data Quality & Documentation Lead):

- **Task:** Writes tests to check if data is clean, helps write the final project report, and prepares the GitHub README.

	Assigned Role	Main Responsibilities & Tasks	Expected Deliverables
Data Engineer 1	**Pipeline & Orchestration Lead**	Set up Apache Airflow environment; build DAG workflow; configure execution schedules, retry rules (retries=3), and error logging.	• dags/ecommerce_etl_dag.py • Screenshots of successful Airflow runs
Data Engineer 2	**Data Ingestion Specialist**	Write Python extraction scripts to pull product data from the web API and ingest raw CSV/JSON customer/order files into the landing folder.	• src/ingest_api.py • src/ingest_files.py • data/raw/ sample files

Data Engineer 3	**Data Transformation Lead**	Write Pandas cleaning scripts; remove duplicate records, impute/handle missing values, and standardize date and currency formats.	• src/transform.py • data/processed/ sample files
Data Engineer 4	**Database & SQL Specialist**	Design database layout (Star Schema); write SQL scripts to create database tables, indexes, primary/foreign keys, and analytical SQL views.	• sql/schema.sql (Tables/Indexes) • sql/views.sql (Analytical Queries) • docs/ERD_Diagram.png
Data Engineer 5	**Data Quality & Documentation Lead**	Write automated data assertion scripts (checking for nulls/duplicates); write technical documentation and assemble the final GitHub repository README.	• src/data_checks.py • README.md • Final Project Report / Slides

Risk Assessment & Mitigation Plan

Risk (Problem)	Impact / Severity	Why It Matters	Solution (Mitigation Strategy)
1. Web API Outage or Connection Failures	**High**	If the external product API goes down or network drops, the data extraction step fails.	**Automatic Retries & Local Cache:** Configure Python to retry 3 times with a delay. If the API stays down, load a saved local JSON backup file so the pipeline can keep running.

2. Missing or Bad Data in Source Files	**High**	Raw CSV/JSON files might contain blank fields, wrong dates, or corrupt text that breaks database loads.	**Data Validation Checks:** Use a Python script to check column names, date formats, and empty fields before inserting records into PostgreSQL.
3. Duplicate Records	**Medium**	Receiving duplicate order entries distorts total sales calculations in business reports.	**Pandas Deduplication:** Execute df.drop_duplicates() on unique record IDs (e.g., order_id) during the transformation step.
4. Team Skill Gaps / Learning Curve	**Medium**	Team members may struggle with complex cloud services or advanced tools within the short 5-week deadline.	**Simple Core Architecture:** Focus on fundamental Python (Pandas) and standard SQL. Keep tools local using PostgreSQL and Apache Airflow without adding unnecessary complexity.

KPIs (Key Performance Indicators)

Pipeline Reliability (Uptime Rate):

- The automated pipeline should run on schedule without stopping or needing manual bug fixes.
- **Target: 99%** successful scheduled runs in Apache Airflow.

Data Cleanliness & Accuracy Rate:

- All raw data errors must be cleaned during the transform step before reaching the warehouse.
- **Target: 0%** duplicate rows and **0%** invalid/unhandled missing primary keys in the final database.

Pipeline Processing Latency (Speed):

- Ingestion, cleaning, testing, and database loading must finish quickly.
- **Target:** Total execution time under **15 minutes** for a daily batch load of up to 100,000 records.

Analytical Query Response Time:

- Database indexes on frequently queried columns (such as customer_id and order_date) ensure reports load instantly for business analysts.

- **Target:** SQL view queries return results in **less than 2 seconds**.

Code Test Pass Rate:

- Every dataset must pass primary key uniqueness and null-value checks before the Airflow pipeline marks a run as "Success."
- **Target: 100%** passing rate on automated data quality assertion scripts.

2. Literature Review

• Feedback & Evaluation

Feature / Approach	**1. Manual Excel / CSV Processing**	**2. Basic Python Scripts (Cron Job)**	**3. Automated Pipeline (Our Project)**
Description	Analysts manually download CSV files and merge them in Excel.	Simple Python scripts scheduled using basic server cron tasks.	Modular Python ETL + PostgreSQL Star Schema + Apache Airflow DAG.
Data Cleaning	Manual finding and deleting of bad rows.	Basic code cleaning, but hard to catch new errors.	Automated schema checks, deduplication, and quality gates.
Automation	None (100% manual effort).	Partial automation; fails quietly if internet drops.	Full automation with automated retries, schedule logs, and monitoring.
Database Design	Flat spreadsheet files (messy & slow).	Basic relational tables without clear modeling.	Optimized **Star Schema** (Fact_Sales + 3 Dimension tables).

Pros (Advantages)	• Very easy to start. • No coding required.	• Fast to code initially. • Works for small tasks.	• **Highly reliable & automated**. • Easy for analysts to query. • Robust error tracking.
Cons (Disadvantages)	• High human error. • Cannot scale with big data. • Extremely slow.	• Hard to debug errors. • No visual workflow status. • Pipeline breaks silently.	• Requires setting up Airflow. • Small learning curve for orchestration.

• Suggested Improvements

1. **Automation & Self-Healing:** Unlike simple scripts that stop on network errors, our pipeline uses **Apache Airflow** to automatically retry up to 3 times before failing.
2. **Quality-First Ingestion:** Bad rows and duplicate orders are stopped at a **Data Quality Check gate** before entering the database, keeping reports clean.
3. **Analyst-Friendly Warehouse:** Storing data in a **Star Schema** (`Fact_Sales`, `Dim_Customer`, `Dim_Product`, `Dim_Date`) allows business users to write fast SQL queries in seconds.

3. Requirements Gathering

• Stakeholder Analysis

1. Business Data Analysts (Users):

- **Who they are:** The team creating daily sales reports for business managers.
- **Their needs:** Clean, accurate, central data in PostgreSQL so they don't have to clean raw CSV files manually.

2. Data Engineering Team (Our Team):

- **Who we are:** The creators and maintainers of the data pipeline.
- **Our needs:** Modular Python code, easy Airflow task logs, and clear documentation so we can build and explain the project without confusion.

3. Course Instructor / Evaluator:

- **Who they are:** The professor reviewing our technical implementation.
- **Their needs:** A complete working pipeline that follows best practices (Star Schema, Airflow DAG, error handling, data quality checks).

• User Stories & Use Cases

1. **As a Data Analyst,** I want all customer, order, and product records stored in one relational database so that I can calculate total monthly sales using simple SQL queries.
2. **As a Data Engineer,** I want Airflow to log every pipeline step so that I can quickly locate errors if an API connection fails.
3. **As a Business Manager,** I want pre-made SQL views (`view_daily_sales`) so I can see real-time performance without writing complex code.

Use Case: Daily Automatic Pipeline Execution

- **Primary Actor:** Apache Airflow Scheduler
- **Trigger:** Scheduled daily timer or manual execution click in Airflow.
- **Step-by-Step Flow:**
    1. **Fetch:** Python downloads product details from the web API and reads raw customer/order CSV and JSON files.
    2. **Clean:** Python removes duplicate order IDs, fills empty values, and formats all dates as `YYYY-MM-DD`.
    3. **Verify:** A validation check ensures zero blank primary keys exist before saving.
    4. **Load:** Clean data is inserted into the PostgreSQL Star Schema (`Fact_Sales` and Dimension tables).
    5. **Finish:** Airflow logs a "SUCCESS" status message.

• Functional Requirements

1. **FR-01 (Multi-Source Ingestion):** The system MUST pull data from 3 sources: REST API (products), CSV files (customers), and JSON files (orders).

2. **FR-02 (Data Transformation & Cleaning):** The system MUST remove duplicate rows, handle blank fields, and convert date strings into standard formats using Pandas.
3. **FR-03 (Star Schema Relational Database):** The system MUST organize target data in PostgreSQL using a Star Schema structure:
   `Fact_Sales`
   `Dim_Customer`
   `Dim_Product`
   `Dim_Date`
4. **FR-04 (Analytical SQL Views):** The system MUST create pre-defined SQL views (`view_daily_sales`, `view_top_customers`) for business reports.
5. **FR-05 (Automated Orchestration):** The system MUST schedule and execute tasks in order using Apache Airflow.

• Non-functional Requirements

1. Performance:

- The entire pipeline MUST process up to 100,000 raw records in **under 15 minutes**.
- Pre-built SQL views MUST return query results in **less than 2 seconds**.

2. Reliability & Safety:

- If the API fails, the ingestion script MUST retry **3 times** before reporting an error.
- The final database tables MUST contain **0 duplicate rows** and **0 orphan foreign keys**.

3. Usability & Maintainability:

- All Python modules must follow standard PEP 8 naming so any team member can read and edit the code.
- The whole system should be deployable locally using simple execution commands.